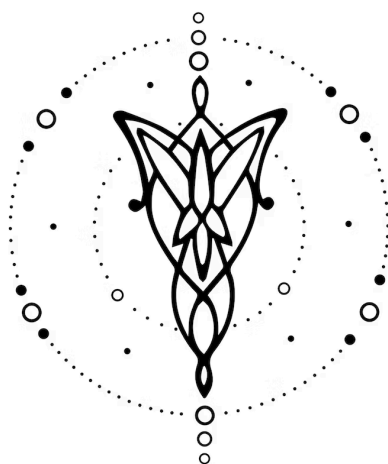




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

1 de Abril de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Lautaro Ezequiel	Domínguez	65.723	ladominguez@itba.edu.ar
Felipe	Matich	65.022	fmatich@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/Lautaro-Dominguez/TPE-TLA>.

3. Dominio

Desarrollar un lenguaje que permita modelar, analizar y simular finanzas personales a través de la definición de ingresos, gastos, activos, deudas y objetivos financieros. El lenguaje debe permitir representar distintos flujos de dinero, calcular métricas como el ingreso neto, patrimonio total y tasa de ahorro, así como definir reglas que automaticen decisiones financieras en base al estado económico del usuario.

Para ello, el lenguaje proveerá construcciones que permitan declarar distintas fuentes de ingreso con su periodicidad, registrar gastos categorizados, representar el patrimonio y las deudas, y establecer objetivos financieros con montos y fechas límites. Además, se podrán definir variables que modifiquen el comportamiento del sistema, como la reasignación de gastos o el ajuste de estrategias de ahorro.

Con el fin de reducir la complejidad del proyecto, el sistema operará sobre un modelo simplificado de finanzas personales, sin interacción con sistemas externos ni datos reales. Todas las operaciones se ejecutarán dentro del contexto del programa, sin conexión a mercados financieros, entidades bancarias o fuentes de datos. Todo será tratado como datos estáticos definidos por el usuario.

La implementación de este lenguaje permitirá simular distintos escenarios financieros personales, analizar el impacto de decisiones económicas y automatizar estrategias de administración del dinero. De esta manera, se facilita la experimentación y comprensión sin incurrir en riesgos reales.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán definir una o varias fuentes de ingreso, donde cada uno tendrá un identificador, un monto y una periodicidad (mensual, semanal, etc.).
- (II). Se podrán registrar uno o varios gastos, cada uno con un identificador, un monto, una periodicidad y una categoría asociada.
- (III). Se podrán definir categorías de gastos, permitiendo organizar los mismos de forma jerárquica.
- (IV). Se podrán representar activos financieros, donde cada activo tendrá un identificador y un valor asociado.
- (V). Se podrán representar deudas, donde cada una tendrá un identificador, un saldo pendiente y opcionalmente una tasa de interés y un pago mínimo.
- (VI). Se podrán definir objetivos financieros, donde cada uno tendrá un identificador, un monto objetivo y una fecha límite.
- (VII). Se podrán declarar variables derivadas, permitiendo calcular métricas como ingreso neto, patrimonio total o tasa de ahorro.
- (VIII). Se proveerán operaciones aritméticas básicas como +, -, * y /.
- (IX). Se proveerán operadores relacionales como <, >, =, ≠, ≤ y ≥.

- (X). Se podrán definir acciones que modifiquen valores existentes, como la reducción de un gasto en un cierto porcentaje o la asignación de un porcentaje del ingreso a ahorro.
- (XI). Se permitirá el uso de porcentajes como tipo de dato para definir proporciones en asignaciones y reglas.
- (XII). Se permitirá trabajar con valores temporales básicos, como fechas y periodicidades, para modelar eventos recurrentes y objetivos financieros.
- (XIII). Todas las operaciones se ejecutarán dentro de un entorno simulado, sin interacción con sistemas externos ni datos reales.
- (XIV). Se permitirá el manejo de distintos tipos de monedas (ARS, USD, entre otros).
- (XV). Se permitirá generar gráficos con la información provista (por ejemplo, un gráfico circular sobre el porcentaje de gastos en cada categoría) y exportar datos en archivos .csv para luego seguir trabajando en otras aplicaciones (como por ejemplo Microsoft Excel).

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que calcule los ingresos y egresos totales de una cuenta bancaria.
- (II). Un programa que calcule la totalidad de gastos en una categoría determinada.
- (III). Un programa que determine en qué categoría gasta más un usuario.
- (IV). Un programa que calcule la cantidad de dinero disponible para ahorrar en un mes determinado.
- (V). Un programa que devuelva ciertos valores en otra moneda distinta a la originalmente indicada.
- (VI). Un programa que genere un gráfico circular con los porcentajes de gasto en cada categoría.
- (VII). Un programa que determine la viabilidad o no de acumular cierto ahorro al cabo de un tiempo determinado, tomando en cuenta valores históricos.
- (VIII). Un programa que permita generar un plan de pago para una deuda en cierta cantidad de cuotas, teniendo en cuenta valores históricos de ahorro.
- (IX). Un programa que modifique el valor de algún dato definido con antelación (por ejemplo: la factura de luz ya no es mensual si no bimestral, por lo que no tendré ese egreso con la misma periodicidad).
- (X). Un programa que genere un archivo .csv con los datos históricos de ahorro mensual.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que declare un ingreso con valor negativo.
- (III). Un programa que determine los gastos totales en una categoría inexistente.
- (IV). Un programa que intente modelar dos variables con el mismo nombre.
- (V). Un programa que intente operar entre tipos de datos incompatibles.

6. Ejemplos

En el **Cód. (6.1)**, se puede ver la creación de variables correspondientes a ingresos y egresos.

```
// Crear una variable que modele un sueldo de 1500000 pesos argentinos:
income sueldo as{
  value = 1500000.00,
  currency = ARS,
  periodicity = monthly
};

// Crear una variable que modele un gasto de 50000 pesos argentinos en la
factura de electricidad:
expenses facturaDeEdenor as{
  value = 50000.00,
  currency = ARS,
  periodicity = monthly,
  category = "Facturas"
};
```

Código 6.1: Creación de variables correspondientes a un ingreso y un egreso.

En el **Cód. (6.2)**, se procede a calcular una variable derivada (IVA pagado) con respecto al total de los gastos efectuados para un periodo determinado..

```
//Calcula el valor del IVA multiplicando a totalExpenses (suma de todos los
//expenses)entre el 21 de enero y el 21 de febrero por 21%
derivatedData IVA = totalExpenses as{
  from "21-01-2026",
  up "21-02-2026"
} * 21%;
```

Código 6.2: Cálculo de IVA pagado haciendo uso de una variable de tipo derivatedData.